Working with Branches

- GIT branch is a separate line of development in a repository. Branches allow developers to work on features, fixes, or experiments without directly changing the main development line.
- A branch can be created from an existing point in the project's history. Developers can work on the branch, create commits, and later merge the changes into another branch.

Why Branches are Used

- To develop new features separately.
- To fix bugs without disturbing the main development line.
- To allow multiple developers to work on different tasks.
- To test or experiment with changes safely.
- To organize project development.
- To combine completed work using merging.

Understanding the Main Branch

The main branch is commonly used as the primary branch of a repository. Depending on the repository or project, it may be named main or another name.

The important idea is that developers can create separate branches from the main development line and work independently.

```
main
  |
  +------ feature-login
  |
  +------ feature-payment
```

Checking the Current Branch

Command

```
git branch
```

Purpose of the Command

- The git branch command displays local branches. The current branch is normally indicated with an asterisk (*).

Example

```
git branch
```

```
* main
  feature-login
  feature-payment
```

In this example, the asterisk indicates that main is the current branch.

Creating a New Branch

Command

```
git branch feature-login
```

Purpose of the Command

- Creates a new branch named feature-login without automatically switching to it.

Example

```
git branch feature-login
git branch
```

After creation, the new branch appears in the branch list, but the current branch remains unchanged.

Switching to a Branch

Command

```
git switch feature-login
```

Purpose of the Command

- Switches the working directory to the specified existing branch.

Example

```
git switch feature-login
git branch
```

The asterisk now appears next to feature-login.

Creating and Switching to a Branch

Command

```
git switch -c feature-login
```

Purpose of the Command

- Creates a new branch and switches to it in one command.

Example

```
git switch -c feature-login
git branch
```

This is a convenient command when starting work on a new feature.

Note: Alternative Command

```
git checkout -b feature-login
```

This older command can also create and switch to a branch. For new classroom examples, git switch is clearer because it is specifically designed for branch switching.

Viewing Branches

Local Branches

```
git branch
```

Purpose: Displays local branches.

Branches with More Information

```
git branch -v
```

Purpose: Displays local branches with information about their latest commits.

Working with a Branch

Once a developer switches to a branch, changes can be made and committed normally.

```
git switch feature-login

# Modify project files

git status
git add .
git commit -m "Added login feature"
```

The commit is created on the current branch. The main branch is not automatically updated by this commit.

Note: Branch and Commit Relationship

Each branch points to a line of commits. When a new commit is created on a branch, that branch moves forward to include the new commit.

```
main
A --- B

feature-login
A --- B --- C --- D
```

Here, commits C and D belong to the feature-login development line. The main branch still points to B until the feature branch is merged.

Merging Branches

Merging is the process of combining changes from one branch into another branch.

Basic Workflow

```
git switch main
git merge feature-login
```

Purpose of git merge

The git merge command combines the changes from the specified branch into the currently switched branch.

Example

```
git switch main
git merge feature-login
```

Here, feature-login is merged into main. The developer must first switch to the branch that should receive the changes.

Note:

- The branch being merged into is the current branch.
- The branch named in git merge is the source of the changes.
- After a successful merge, the changes from the source branch become part of the target branch.

Renaming a Branch

Command

```
git branch -m old-name new-name
```

Purpose of the Command

Renames a local branch.

Example

```
git branch -m feature-login login-feature
```

Deleting a Branch

Command

```
git branch -d feature-login
```

Purpose of the Command

- Deletes the specified local branch when its changes have been merged or Git considers it safe to delete.

Example

```
git switch main
git branch -d feature-login
```

A branch should normally be deleted after its work has been merged and is no longer required.

Note: Force Deleting a Branch

```
git branch -D feature-login
```

Purpose: Force deletes the local branch. This should be used carefully because it can remove a branch that contains unmerged work.